PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies with Algorithms, Pseudocode, Python Programs, Test Cases & Conclusions

FIRST-YEAR B.TECH • PYTHON

NAME: T. Sai Siri Shanmuka

ROLL NO: 26000137 | SECTION: CSE-2

Prepared from the supplied Python Activity-Based Learning case-study PDF

Selected Case Studies

Chosen from the uploaded PDF; beginner-friendly examples using the assigned syllabus.

- 11 Examination Result & Grace Marks
- 12 Attendance Recovery
- 13 ATM Transaction Simulation
- 17 Student Performance Trends
- 19 Shopping Budget Constraint
- 27 Numeric Pattern Generator
- 29 Password Policy Validation
- 42 Student Marks Matrix
- 49 Tuple-Based Product Catalogue
- 57 Smart Canteen Order & Stock

Syllabus & Solution Method

Every solution stays within the concepts listed in the assignment.

Assigned concepts used

- Variables, data types and operators
- if / elif / else conditions
- for and while loops
- Strings and string operations
- Lists and nested lists
- Tuples, tuple operations and unpacking
- Pattern generation
- Algorithms, flowcharts and pseudocode

Standard workflow

1. Understand problem
2. Identify inputs and outputs
3. Write algorithm / pseudocode
4. Draw flowchart
5. Code and test
6. Record output and conclusion

Case 1: Examination Result & Grace-Mark Validation

PDF topic #11 • Problem definition and project planning

Problem statement

Calculate a student's total and percentage, check subject pass marks, and apply a limited grace-mark policy without exceeding the maximum marks.

Objectives

- Calculate total and percentage
- Check each subject's pass requirement
- Apply grace marks only when allowed
- Assign a final result

Assumptions

- Three subjects, each out of 100
- Pass mark is 35 in each subject
- At most 5 grace marks may be added to one failing subject
- Grace marks cannot raise a subject above 100

Algorithm

- Input three subject marks
- Validate marks are between 0 and 100
- Find subjects below 35
- If exactly one subject is short by at most 5, add the required grace marks
- Calculate total and percentage
- Display PASS or FAIL

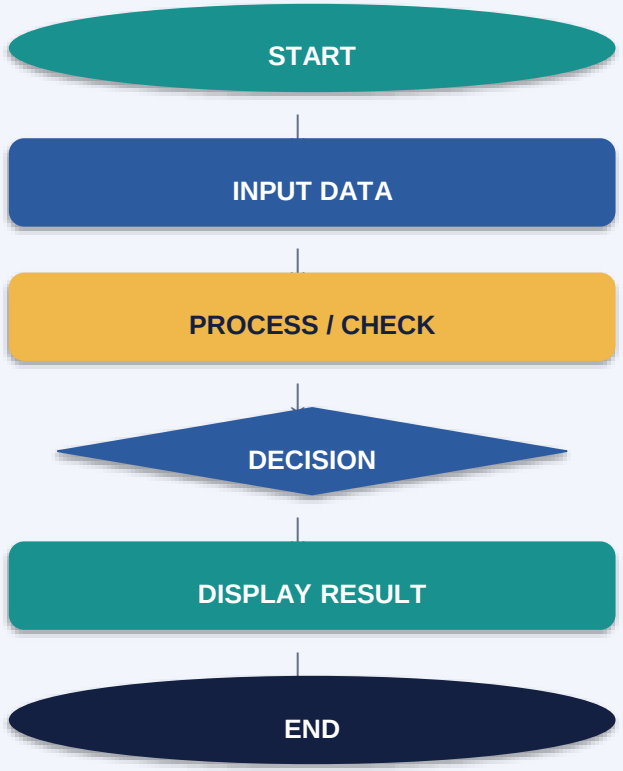
Case 1: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
INPUT marks for 3 subjects
IF any mark < 0 or > 100: show invalid input
ELSE:
  find subjects with marks < 35
  IF one failed subject and shortfall <= 5:
    add shortfall as grace marks
  calculate total and percentage
  IF all marks >= 35: display PASS
  ELSE: display FAIL
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 1: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
marks = []
for i in range(3):
    m = int(input("Enter subject mark: "))
    if m < 0 or m > 100:
        print("Invalid mark")
        break
    marks.append(m)
else:
    failed = [i for i in range(3) if marks[i] < 35]
    if len(failed) == 1 and 35 - marks[failed[0]] <= 5:
        marks[failed[0]] = 35
        print("Grace marks applied")
    total = sum(marks)
    print("Final marks:", marks)
    print("Total:", total)
    print("Percentage:", total / 3, "%")
    if all(m >= 35 for m in marks):
        print("Result: PASS")
    else:
        print("Result: FAIL")
```

Case 1: Testing & Conclusion

Sample test case and expected output

Test case

Input: 40, 32, 60

Expected: grace marks applied; final marks [40, 35, 60]; total 135; 45.0%; PASS

Expected output

grace marks applied; final marks [40, 35, 60]; total 135; 45.0%; PASS

Conclusion

The program validates marks, applies a defined grace rule, and determines the final result.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 2: Attendance Recovery Computation

PDF topic #12 • Problem definition and project planning

Problem statement

Calculate current attendance and determine the minimum consecutive classes a student must attend to reach an eligibility threshold.

Objectives

- Calculate attendance percentage
- Check eligibility
- Count required future attended classes

Assumptions

- Attended and conducted classes are non-negative
- The student attends every future class
- Eligibility threshold is 75%

Algorithm

- Input attended and total classes
- Validate values
- Calculate current percentage
- If eligible, display eligible
- Otherwise add one attended class at a time until percentage reaches 75%

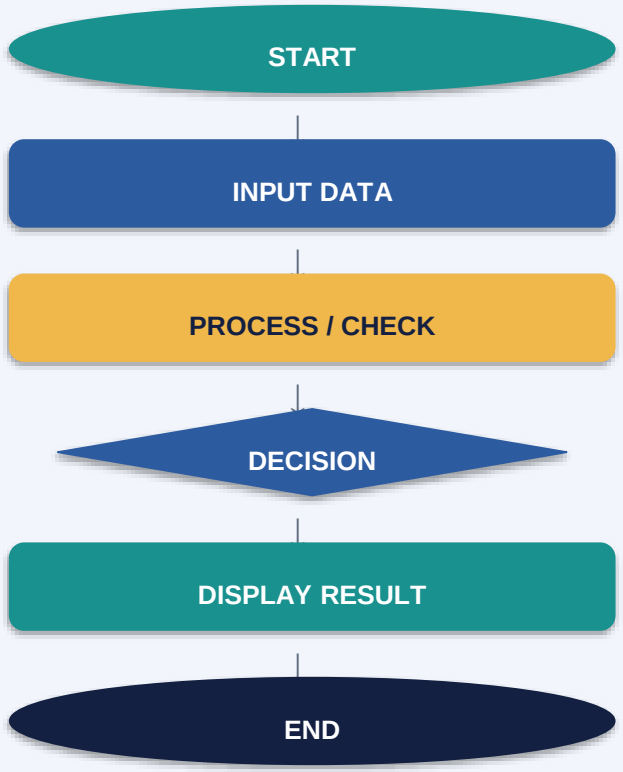
Case 2: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
INPUT attended, total
IF total <= 0 OR attended < 0 OR attended > total:
invalid
ELSE:
  percent = attended / total * 100
  future = 0
  WHILE (attended + future) / (total + future) * 100
< 75:
    future = future + 1
  DISPLAY percent and future
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 2: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
attended = int(input("Classes attended: "))
total = int(input("Total classes held: "))
if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance data")
else:
    percent = attended / total * 100
    future = 0
    while (attended + future) / (total + future) * 100 < 75:
        future += 1
    print("Current attendance:", round(percent, 2), "%")
    if percent >= 75:
        print("Already eligible")
    else:
        print("Consecutive classes to attend:", future)
```

Case 2: Testing & Conclusion

Sample test case and expected output

Test case

Input: attended = 60, total = 100

Expected: current attendance 60.0%; classes to attend consecutively = 60

Expected output

current attendance 60.0%; classes to attend consecutively = 60

Conclusion

A loop models future attendance and finds the minimum classes needed for the threshold.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 3: ATM Transaction Sequence Simulation

PDF topic #13 • Problem definition and project planning

Problem statement

Process a sequence of deposits and withdrawals, maintain a simulated balance, reject invalid transactions, and summarize the result.

Objectives

- Update balance safely
- Reject invalid amounts and overdrafts
- Count successful deposits and withdrawals

Assumptions

- Opening balance is 1000
- Enter D for deposit and W for withdrawal
- Each transaction amount must be positive

Algorithm

- Set opening balance
- Repeat for the requested number of transactions
- Read type and amount
- Apply valid deposit or withdrawal
- Reject invalid operation or insufficient balance
- Display final balance and counts

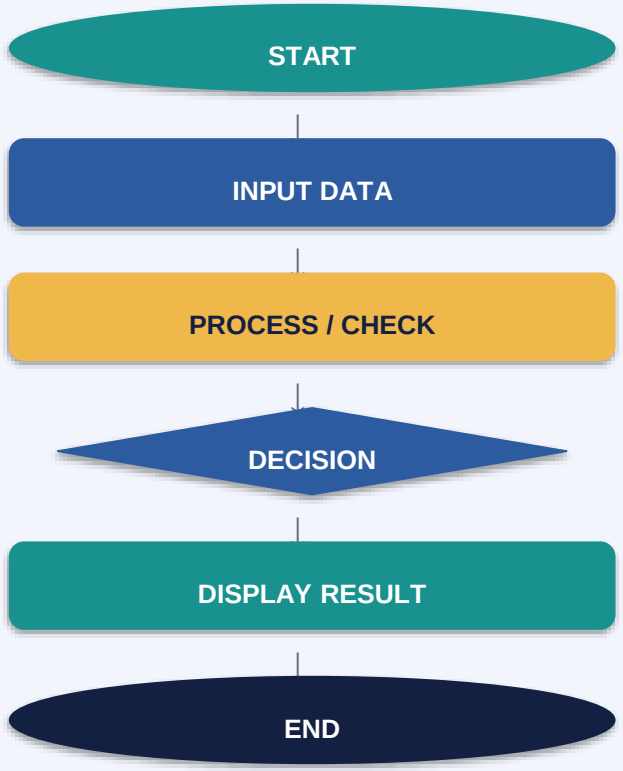
Case 3: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
balance = 1000
INPUT number of transactions
REPEAT that many times:
  INPUT type, amount
  IF amount <= 0: reject
  ELSE IF type is D: add amount; count deposit
  ELSE IF type is W AND amount <= balance: subtract;
count withdrawal
  ELSE: reject
DISPLAY balance and counts
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 3: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
balance = 1000
deposits = 0
withdrawals = 0
n = int(input("Number of transactions: "))
for i in range(n):
    kind = input("D for deposit, W for withdrawal: ").upper()
    amount = float(input("Amount: "))
    if amount <= 0:
        print("Rejected: amount must be positive")
    elif kind == "D":
        balance += amount
        deposits += 1
    elif kind == "W" and amount <= balance:
        balance -= amount
        withdrawals += 1
    else:
        print("Rejected: invalid type or insufficient balance")
print("Final balance:", balance)
print("Successful deposits:", deposits)
print("Successful withdrawals:", withdrawals)
```

Case 3: Testing & Conclusion

Sample test case and expected output

Test case

Opening balance 1000; D 500; W 200; W 2000

Expected: final balance 1300; successful deposits 1; successful withdrawals 1; last withdrawal rejected

Expected output

final balance 1300; successful deposits 1; successful withdrawals 1; last withdrawal rejected

Conclusion

The transaction loop updates the balance and enforces basic banking rules.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 4: Student Performance Trend Analysis

PDF topic #17 • Problem definition and project planning

Problem statement

Compare scores across assessments, calculate changes, and identify subjects with consistently weak results.

Objectives

- Store marks in lists
- Calculate score changes
- Identify weak subjects

Assumptions

- Each subject has two assessment marks
- Marks are from 0 to 100
- A score below 40 is considered weak for this example

Algorithm

- Input subject names and two scores per subject
- Store names and scores in lists
- Calculate second score minus first score
- Label improvement, decline, or no change
- Flag second scores below 40

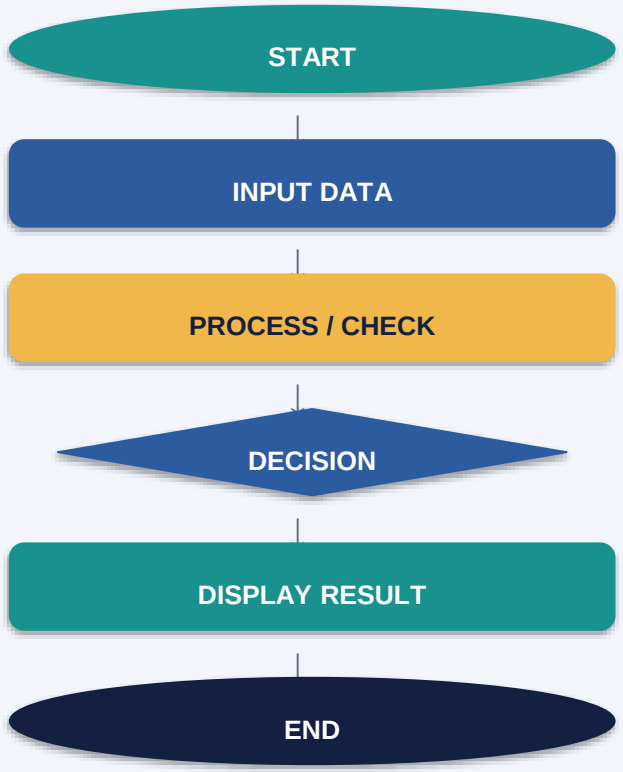
Case 4: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
FOR each subject:
  INPUT name, first_score, second_score
  change = second_score - first_score
  IF change > 0: improved
  ELSE IF change < 0: declined
  ELSE: unchanged
  IF second_score < 40: mark weak
DISPLAY subject-wise analysis
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 4: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
subjects = ["Python", "Maths", "English"]
first = [55, 38, 70]
second = [68, 35, 70]
for i in range(len(subjects)):
    change = second[i] - first[i]
    if change > 0:
        trend = "Improved"
    elif change < 0:
        trend = "Declined"
    else:
        trend = "No change"
    print(subjects[i], "change:", change, "-", trend)
    if second[i] < 40:
        print("  Needs attention")
```

Case 4: Testing & Conclusion

Sample test case and expected output

Test case

Expected: Python +13 Improved; Maths -3 Declined and Needs attention; English 0 No change

Expected output

Python +13 Improved; Maths -3 Declined and Needs attention; English 0 No change

Conclusion

Lists and loops make it easy to compare student results across assessments.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 5: Multi-Product Shopping Budget Constraint

PDF topic #19 • Problem definition and project planning

Problem statement

Calculate a shopping basket total and determine whether it fits a fixed budget.

Objectives

- Store product names, prices, and quantities
- Calculate line totals
- Report budget difference

Assumptions

- Prices and quantities are positive
- Budget is 1500
- No discounts are applied in this basic version

Algorithm

- Read product count
- For each product input name, price, quantity
- Calculate price multiplied by quantity
- Add to total
- Compare total with budget
- Display remaining amount or overspending

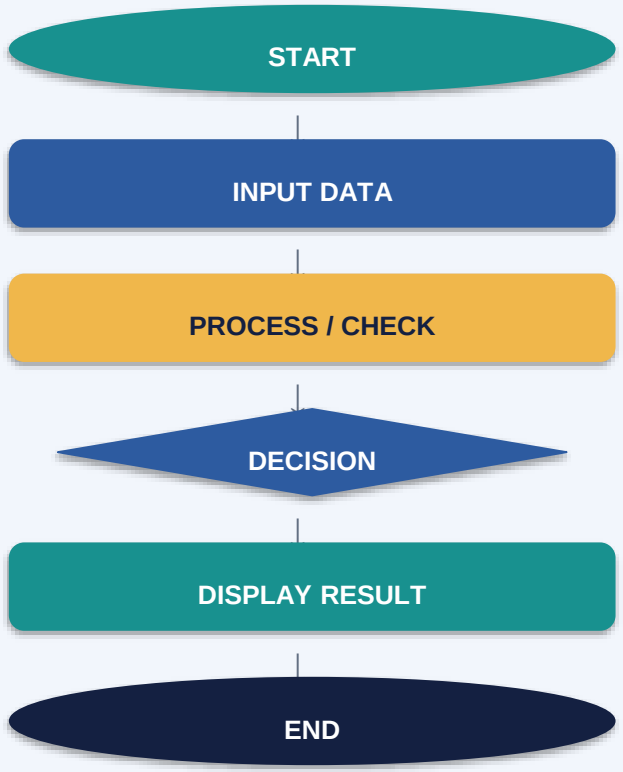
Case 5: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
total = 0
FOR each product:
  INPUT name, price, quantity
  line_total = price * quantity
  total = total + line_total
IF total <= budget: show remaining budget
ELSE: show amount over budget
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 5: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
names = []
prices = []
quantities = []
n = int(input("Number of products: "))
for i in range(n):
    name = input("Product name: ")
    price = float(input("Price: "))
    qty = int(input("Quantity: "))
    names.append(name)
    prices.append(price)
    quantities.append(qty)
total = 0
for i in range(n):
    line = prices[i] * quantities[i]
    total += line
    print(names[i], "=", line)
budget = 1500
print("Basket total:", total)
if total <= budget:
    print("Budget remaining:", budget - total)
else:
    print("Over budget by:", total - budget)
```

Case 5: Testing & Conclusion

Sample test case and expected output

Test case

Products: notebook ₹100 × 3; bag ₹900 × 1. Expected total ₹1200; remaining budget ₹300

Expected output

Products: notebook ₹100 × 3; bag ₹900 × 1. Expected total ₹1200; remaining budget ₹300

Conclusion

The program uses lists, multiplication, loops, and conditions to track spending.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 6: Advanced Numeric Pattern Generator

PDF topic #27 • Problem definition and project planning

Problem statement

Generate a centered numeric pyramid using nested loops and validate the requested height.

Objectives

- Practice nested loops
- Generate aligned output
- Reject invalid dimensions

Assumptions

- Height must be between 1 and 9
- Each row displays numbers from 1 up to its row number

Algorithm

- Input height
- Validate height
- For each row from 1 to height
- Print leading spaces
- Print numbers from 1 to row number
- Move to next line

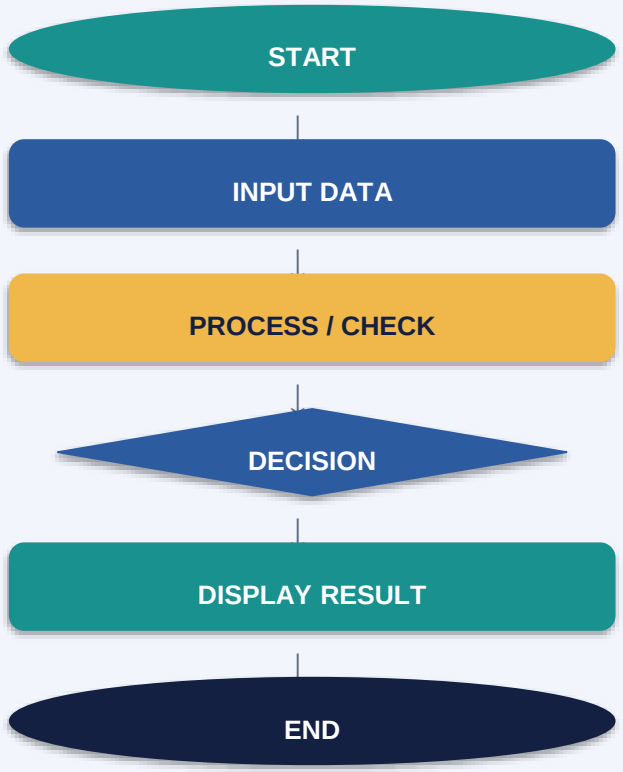
Case 6: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
INPUT height
IF height < 1 OR height > 9: invalid
ELSE:
  FOR row from 1 to height:
    PRINT leading spaces without newline
    FOR number from 1 to row:
      PRINT number without newline
    PRINT newline
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 6: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
height = int(input("Enter pattern height (1-9): "))
if height < 1 or height > 9:
    print("Invalid height")
else:
    for row in range(1, height + 1):
        print(" " * (height - row), end="")
        for number in range(1, row + 1):
            print(number, end="")
        print()
```

Case 6: Testing & Conclusion

Sample test case and expected output

Test case

Input: 4

Output:

```
1
12
123
1234
```

Expected output

Input: 4

Output:

```
1
12
123
1234
```

Conclusion

Nested loops and string repetition produce a simple aligned numeric pattern.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 7: Password Policy Validation Engine

PDF topic #29 • Problem definition and project planning

Problem statement

Check a sample password string against basic length and character-category requirements.

Objectives

- Use string methods
- Check multiple conditions
- Explain whether the sample passes

Assumptions

- Minimum length is 8
- Password must contain an uppercase letter, lowercase letter, and digit
- This is a classroom exercise, not a complete security checker

Algorithm

- Input password
- Check length
- Loop through characters to find uppercase, lowercase, and digit
- Display each condition
- Accept only if all conditions pass

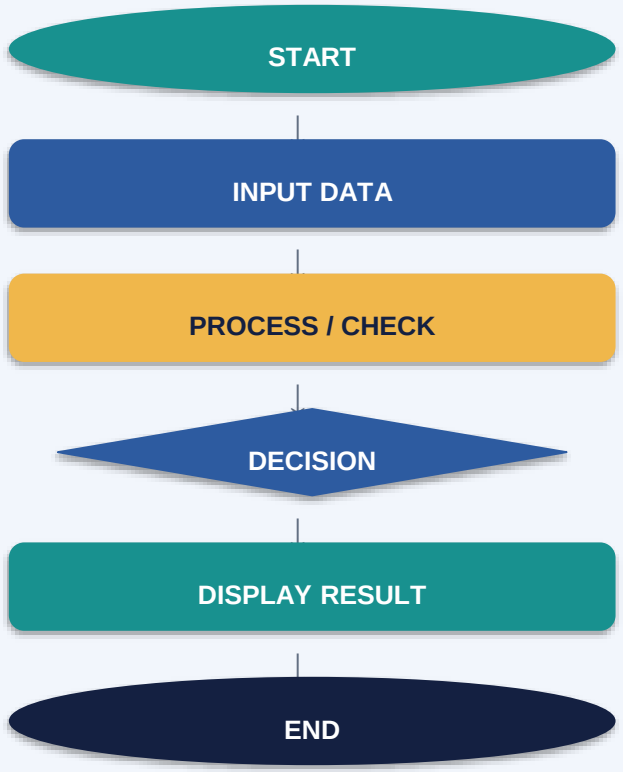
Case 7: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
INPUT password
has_upper = false; has_lower = false; has_digit = false
FOR each character:
  update flags based on character category
IF length >= 8 AND all flags true: valid
ELSE: invalid and show missing conditions
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 7: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
password = input("Enter sample password: ")
has_upper = False
has_lower = False
has_digit = False
for ch in password:
    if ch.isupper():
        has_upper = True
    if ch.islower():
        has_lower = True
    if ch.isdigit():
        has_digit = True
print("Length >= 8:", len(password) >= 8)
print("Has uppercase:", has_upper)
print("Has lowercase:", has_lower)
print("Has digit:", has_digit)
if len(password) >= 8 and has_upper and has_lower and has_digit:
    print("Sample password meets the rules")
else:
    print("Sample password does not meet all rules")
```

Case 7: Testing & Conclusion

Sample test case and expected output

Test case

Input: Python123

Expected: length true; uppercase true; lowercase true; digit true; meets the rules

Expected output

length true; uppercase true; lowercase true; digit true; meets the rules

Conclusion

String operations and Boolean conditions evaluate each password requirement.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 8: Student Marks Matrix Analysis

PDF topic #42 • Problem definition and project planning

Problem statement

Store marks in a nested list and calculate each student's total and average, then identify the highest total.

Objectives

- Represent data as a nested list
- Calculate row totals and averages
- Identify the top-performing row

Assumptions

- Each row contains marks for three subjects
- All marks are from 0 to 100

Algorithm

- Create a nested list of marks
- Loop through each student's row
- Calculate total and average
- Track the highest total
- Display results

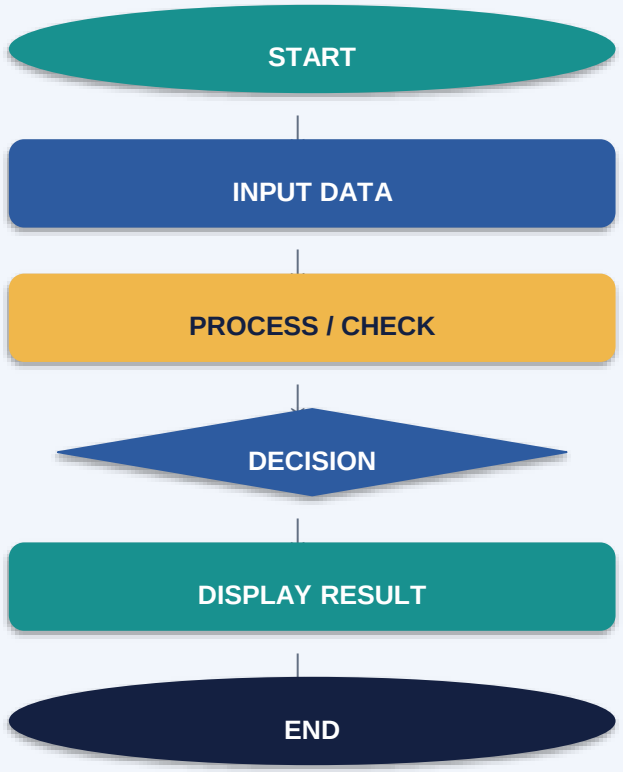
Case 8: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
marks = nested list
best_total = -1
FOR each student row:
  total = sum(row)
  average = total / number of subjects
  DISPLAY total and average
  IF total > best_total: update best_total and best
student
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 8: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
marks = [[78, 85, 90], [88, 76, 92], [67, 80, 74]]
best_total = -1
best_student = 0
for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    print("Student", i + 1, "total:", total, "average:", round(average, 2))
    if total > best_total:
        best_total = total
        best_student = i + 1
print("Highest total:", best_total)
print("Top student:", best_student)
```

Case 8: Testing & Conclusion

Sample test case and expected output

Test case

Expected totals: 253, 256, 221; averages: 84.33, 85.33, 73.67; top student: 2

Expected output

Expected totals: 253, 256, 221; averages: 84.33, 85.33, 73.67; top student: 2

Conclusion

Nested lists represent a marks matrix and allow row-wise analysis.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 9: Tuple-Based Product Catalogue Audit

PDF topic #49 • Problem definition and project planning

Problem statement

Store product details as tuples, calculate inventory value, and identify products that need replenishment.

Objectives

- Represent fixed product records with tuples
- Unpack tuple values
- Calculate stock value and check stock level

Assumptions

- Each tuple contains product name, price, and stock quantity
- Replenishment is needed when stock is below 5

Algorithm

- Create a list of product tuples
- Unpack each tuple
- Calculate price multiplied by stock
- Display inventory value
- Flag stock below 5

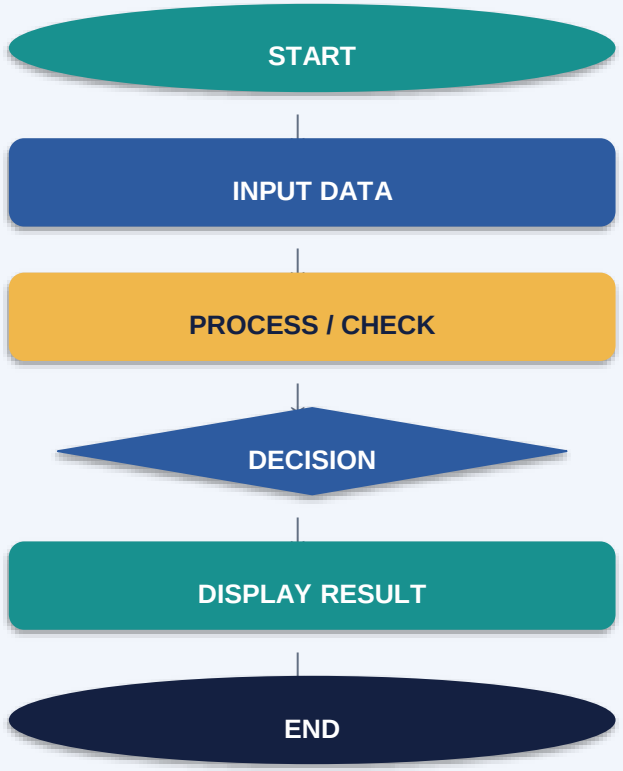
Case 9: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
FOR each tuple (name, price, stock):
  value = price * stock
  DISPLAY name and value
  IF stock < 5: flag for replenishment
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 9: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
products = [("Pen", 10, 20), ("Notebook", 60, 3), ("Folder", 25, 8)]
for name, price, stock in products:
    value = price * stock
    print("Product:", name)
    print("Inventory value:", value)
    if stock < 5:
        print("Replenishment required")
```

Case 9: Testing & Conclusion

Sample test case and expected output

Test case

Expected: Pen value ₹200; Notebook value ₹180 and replenishment required; Folder value ₹200

Expected output

Pen value ₹200; Notebook value ₹180 and replenishment required; Folder value ₹200

Conclusion

Tuples store related product fields, and unpacking makes each record easy to process.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

Case 10: Smart Canteen Order & Stock Management

PDF topic #57 • Problem definition and project planning

Problem statement

Process customer orders, calculate bills, update available stock, reject unavailable quantities, and summarize sales.

Objectives

- Store menu and stock
- Validate order quantities
- Update stock after a successful order
- Calculate sales

Assumptions

- Sample menu has three items
- No tax or discount is applied
- Only available quantities can be sold

Algorithm

- Create item, price, and stock lists
- Input number of order lines
- Read item number and quantity
- Validate item and stock
- Calculate bill and update stock
- Display total sales and remaining stock

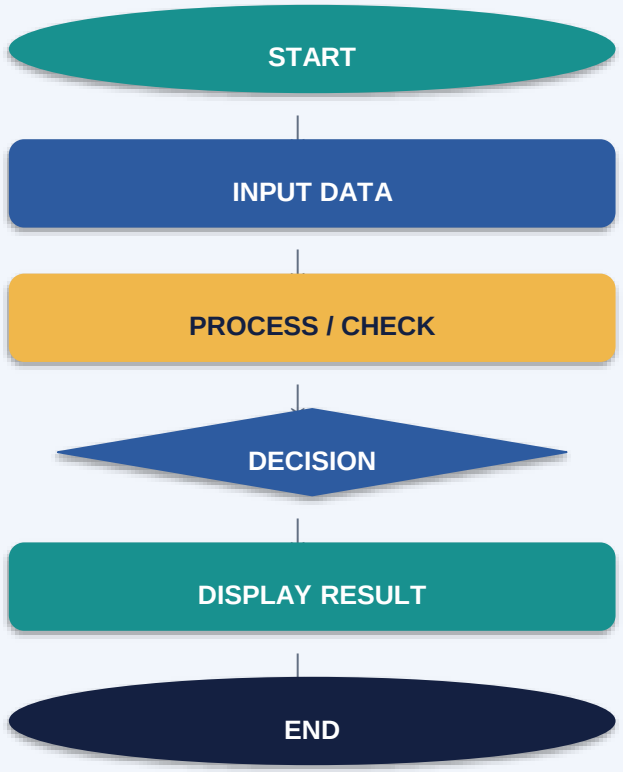
Case 10: Algorithm Design

Pseudocode and flowchart logic

PSEUDOCODE

```
menu, prices, stock = predefined lists
sales = 0
FOR each order:
  INPUT item_number, quantity
  IF item_number invalid OR quantity <= 0: reject
  ELSE IF quantity > stock[item]: reject
  ELSE:
    bill = price[item] * quantity
    sales = sales + bill
    stock[item] = stock[item] - quantity
DISPLAY sales and stock
```

FLOWCHART



Note: The flowchart shows the general control flow. Follow the case-specific pseudocode for exact conditions.

Case 10: Python Source Code

Uses only basic Python concepts from the assigned syllabus.

```
items = ["Idli", "Dosa", "Juice"]
prices = [30, 50, 25]
stock = [20, 15, 10]
sales = 0
n = int(input("Number of order lines: "))
for i in range(n):
    print("Menu:", list(enumerate(items, start=1)))
    item_no = int(input("Choose item number: "))
    qty = int(input("Quantity: "))
    index = item_no - 1
    if index < 0 or index >= len(items) or qty <= 0:
        print("Invalid order")
    elif qty > stock[index]:
        print("Not enough stock")
    else:
        bill = prices[index] * qty
        sales += bill
        stock[index] -= qty
        print("Order bill:", bill)
print("Total sales:", sales)
print("Remaining stock:")
for i in range(len(items)):
    print(items[i], stock[i])
```

Case 10: Testing & Conclusion

Sample test case and expected output

Test case

Order 2 Dosas and 1 Juice. Expected sales ₹125; remaining stock: Idli 20, Dosa 13, Juice 9

Expected output

Order 2 Dosas and 1 Juice. Expected sales ₹125; remaining stock: Idli 20, Dosa 13, Juice 9

Conclusion

The program combines lists, loops, conditions, and arithmetic to model simple canteen operations.

For submission: run the program, capture an actual output screenshot, and add it to the project folder.

PROJECT SUMMARY

What this project demonstrates

- Problem-solving through computational thinking
- Algorithms, pseudocode, and flowchart logic
- Variables, data types, operators, conditions, and loops
- Strings, lists, nested lists, tuples, and tuple unpacking
- Testing with sample inputs and checking expected outputs

10
CASE
STUDIES

Thank you

Reference: Python Activity-Based Learning case-study PDF supplied for this assignment.